M T W T F S S

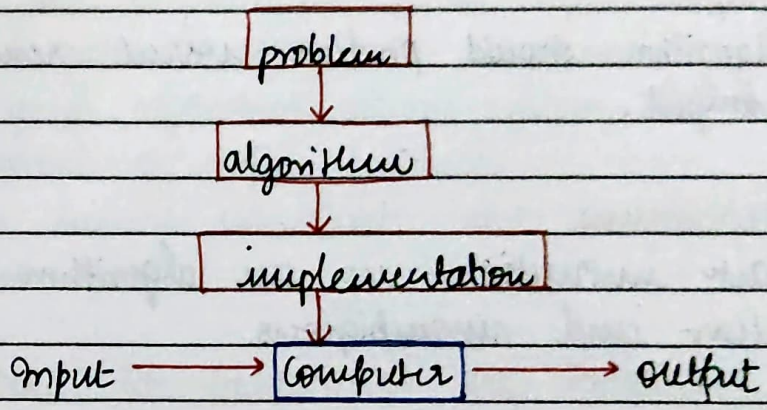
COMPASS

Date :

DESIGN AND ANALYSIS OF ALGORITHMS

ALGORITHM:

An algorithm is a step by step set of instruction to solve a given problem in a finite number of steps by accepting a set of inputs and producing the desired output.



LINEAR / SEQUENTIAL SEARCH:

(is make diagrammatic representation!!)

* Brute force technique.

0	1	2	3	4	5	6
66	11	33	88	44	99	22

ALGORITHM LinearSearch(a[0....n-1], key)
//purpose: Search key item in a given list sequentially
//input: a[0....n-1] ← array in size n
key ← key item to search
//output: if key found return index i otherwise return -1

```
for i ← 0 to n-1 do
    if a[i] == key then
        return i            //key found.
end for
return -1                   //key not found.
```

COMPASS

M T W T F S S

Date :

CHARACTERISTICS OF AN ALGORITHM

1. **Input:**
Each algorithm should have zero (or) more inputs. The range of input for which algorithm work should be satisfied.

2. **Output:**
Algorithm should produce correct result or desired output.

3. **Definiteness:**
Each instruction in an algorithm should be clear and unambigious.

4. **Efficiency:**
Algorithm must run less time and occupy less memory.

5. **Finiteness:**
The algorithm must terminate after finite number of steps.

6. Same algorithm can be represented in several different ways
   - Natural language.
   - Pseudo code
   - Flow chart.

M T W T F S S

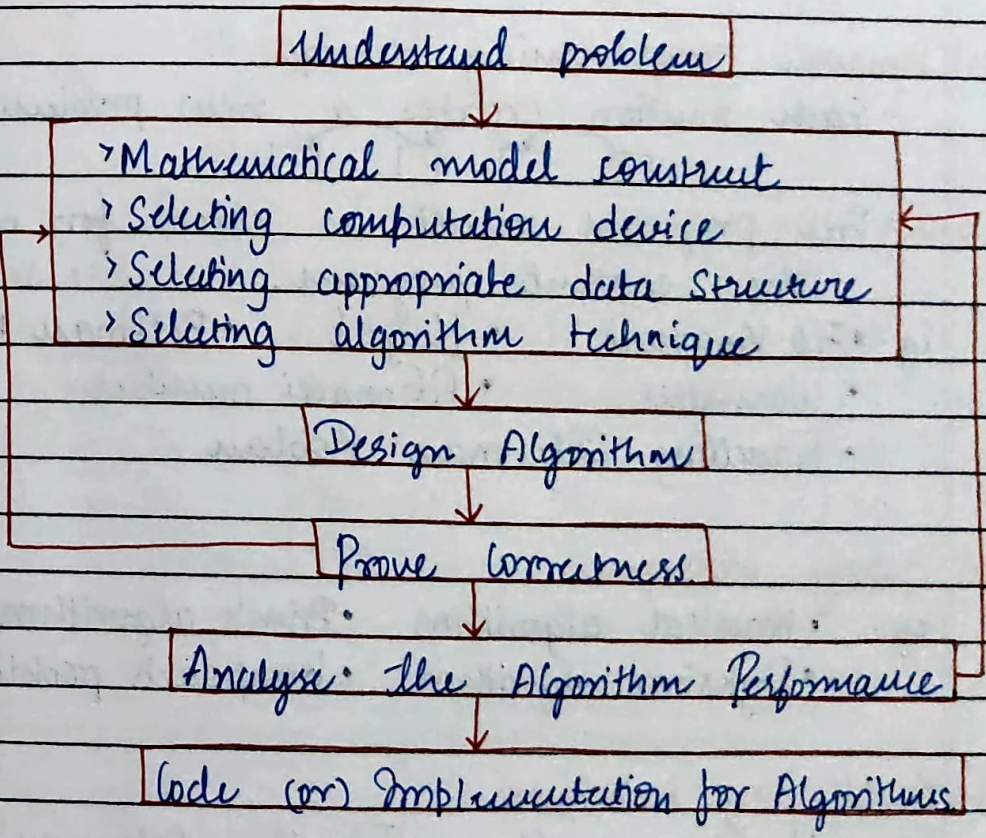
COMPASS

Date :

ROLE OF ALGORITHM COMPUTING

* Algorithm provide a systematic and logic approach to solving problem and form the core of all computer programs
* Problem solving : Algorithm define clear step to solve computational problems efficiently and correctly.
* Efficiency and performance : Good algorithm optimize time and memory usage to improve system performance
* Well designed algorithm allows system to handle large amount of data effectively
* Algorithm ensure consistent and accurate results for same input.

FUNDAMENTAL OF ALGORITHM AND PROBLEM SOLVING



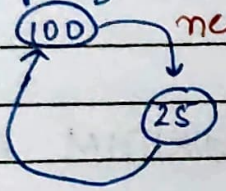
M T W T F S S
Date :

ALGORITHM DESIGN TECHNIQUES

(i) Brute force technique [Straight forward approach]
Eg: • Bubble Sort • Linear Search
• Selection Sort • Matrix Multiplication

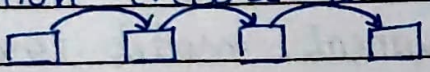
(ii) Divide & Conquer technique (↑ resources)
Eg: • Binary Search • Merge Sort • Quick Sort
• Strassen matrix multiplication

(iii) Decrease & Conquer technique (↓ resources)
Loop the sub-program until solved completely



Eg: • BFS • DFS
• Topological Sorting

(iv) Dynamic programming technique
- Each solution creates a new problem



- Prev program result is used for computing the current program.

Eg: • 1/0 Knapsack • Floyd's • Bellman Ford
• Warshalls • Fibonacci number
• Travelling Salesman Problem

(v) Greedy technique
Eg: • Kruskal algorithm • Prim's algorithm
• Dijkstra algorithm • knapsack problem

(vi) Backtracking technique
Eg: • N-Queen problem • Travelling Salesman Problem
• Chess game • Subset • Graph colouring

M T W T F S S

COMPASS

Date :

(vii) Transform conquer technique:

Eg: • Heap sort • AVL tree

BUBBLE SORT:

ALGORITHM BubbleSort (A[0....n-1])

//purpose: Sort elements

//input: for i=0 to n-2 do

for j=0 to n-2-i

if (A[j+1] < A[j])

swap (A[j+1], A[j])

90	44	70	33	22

Phase i=0

90 → j	44	44	44	44
44 → j+1	90 → j	70	70	70
70	70 → j+1	90 → j	33	33
33	33	33 → j+1	90 → j	22
22	22	22	22 → j+1	90 → maximum

Phase i=1

44 → j	44	44	44
70 → j+1	33 → j	33	33 → j+1
33	70 → j+1	22 → j	44 → j
22	22	70 → j+1	70
90	90	90	90

Phase i=2

44 → j	33	33
33 → j+1	44 → j	22
22	22 → j+1	44
70	70	70
90	90	90

Phase i=3

33 → j	22
22 → j+1	33
44	44
70	70
90	90

//time complexity:

Basic operation: if A[j+1] < A[j]

$$T(n) = c\sum_{i=0}^{n-2}\sum_{j=0}^{n-2-i} 1$$

$$\sum_{lb}^{ub} 1 = ub - lb + 1$$

$$T(n) = c\sum_{i=0}^{n-2}\left((n-2-i)-0+1\right)$$

$$T(n) = c\sum_{i=0}^{n-2}(n-1-i)$$

$$= c\left[(n-1)+(n-2)+(n-3)+\dots+1\right]$$

$$= c\,\frac{(n-1)(n-1+1)}{2} = c\left[\frac{n(n-1)}{2}\right]$$

formula:

$$1+2+3+\dots+n = \frac{n(n+1)}{2}$$

$$T(n) = \frac{cn^2}{2} - \frac{cn}{2} = T(n) \in O(n^2)$$

↳ worst case time complexity

for: n=5

$$\Rightarrow \frac{25}{2} - \frac{5}{2} = \frac{20}{2} = 10$$

for random number generation:

a[i] = rand()%500;

for time complexity:

clock_t start, end;

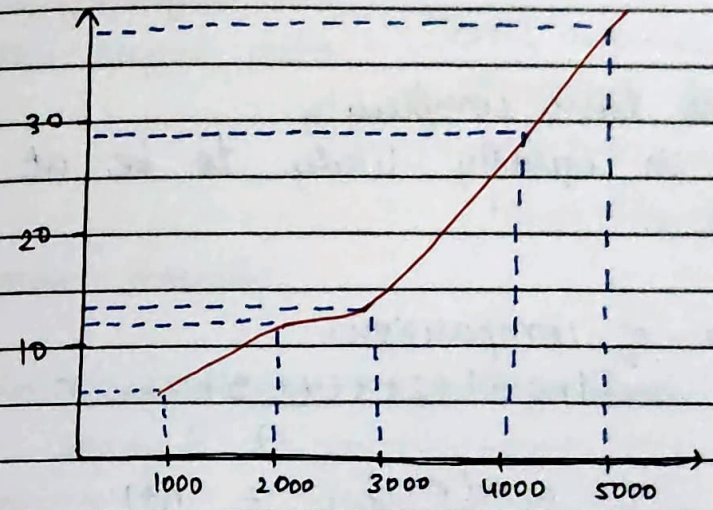
M T W T F S S

COMPASS

Date :

//graph:

n size	time taken
1000	~~5.846~~ 0.006040 ≈ 6.040ms
2000	0.011742 ≈ 11.742ms
3000	0.012801 ≈ 12.801ms
4000	0.029591 ≈ 29.591ms
5000	0.038199 ≈ 38.199 ms.



ANALYSIS OF ALGORITHM PERFORMANCE.

> Performance of an algorithm is measured in two factors
  - (i) time complexity (or) time efficiency
  - (ii) Space complexity (or) space efficiency.

(i) Time complexity:

Time complexity / efficiency of an algorithm is a measure of the amount of time an algorithm takes to execute, expressed as a function of input size.

Eg: Linear Search.

Best case : (key found at 1st position) $= T(n) = \Omega(1)$

Single loop → $T(n) = n$

Double loop → $T(n) = n^2$

M T W T F S S

Date :

Worst case (key found at last position / key not found):

$$T(n) = \sum_{i=0}^{n-1} 1$$

$$\sum_{lb}^{ub} 1 = ub - lb + 1$$

$$= (n-1) - 0 + 1$$

$$= n$$

$$T(n) \in O(n) \rightarrow \text{worst case time complexity.}$$

Avg-case time complexity

Key is equally likely to be at any position

Avg no. of comparisions.

$$T(n) = \frac{1+2+3+4+5+\ldots+n}{n}$$

$$= \frac{n(n+1)}{2} \Big/ n = \frac{n+1}{2}$$

$$\therefore T(n) = \frac{(n+1)}{2} \qquad T(n) \in \Theta(n)$$

- ignore constant parameters.

Dependent on

(i) Computer screen

(ii) Compiler

(iii) Input size

(iv) Choice of Algorithm techniques.

M T W T F S S

COMPASS

Date :

(ii) Space Complexity:

Space complexity of an algorithm is the total amount of memory space required by the algorithm to execute completely, expressed as a function of the input size n.

Total Space Complexity : input space + Auxiliary Space.

Memory required to store input data.

Extra memory required by algorithm during execution → stack memory, variable temporary data.

Eg: Linear Search:-

S(n) = input size + Auxiliary space.

input array A[0.....n-1] = O(n)

variable key = O(1)

S(n) = O(n) + O(1)

ORDER OF GROWTH OF AN ALGORITHM

Describe how fasts its running time grows as the input size n is increased.

Common order of growth:

Bubble sort / Matrix.

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$

$< O(2^n) < O(n!)$

↳ tower of hanoi recursion fibonacci

Matrix Multiplication.

COMPASS

M T W T F S S

Date :

Input Size	O(1)	O(log n)	O(n)	O(n log n)	O(n²)	O(n³)	O(2ⁿ)	O(n!)
1	1	0	1	0	1	1	2	1
2	1	1	2	2	4	8	4	2
4	1							
8	1							
16	1							
32	1	5	32	160	1024	32788	4.29×10^9	2.63×10^3
64	1							

Hashing, array indexing

Binary Search

Linear Search

Merge/ Quick Sort

Bubble Sort / Selection sort

Matrix multiplication Floyd, Warshal

Tower of Hanoi / recursive fibonacci

Travelling Salesman proble

COMPASS

M T W T F S S

Date :

Input Size	O(1)	O(log n)	O(n)	O(n log n)	O(n²)	O(n³)	O(2ⁿ)	O(n!)
1	1	0	1	0	1	1	2	1
2	1	1	2	2	4	8	4	2
4	1							
8	1							
16	1							
32	1	5	32	160	1024	32788	4.29×10^9	2.63×1
64	1							

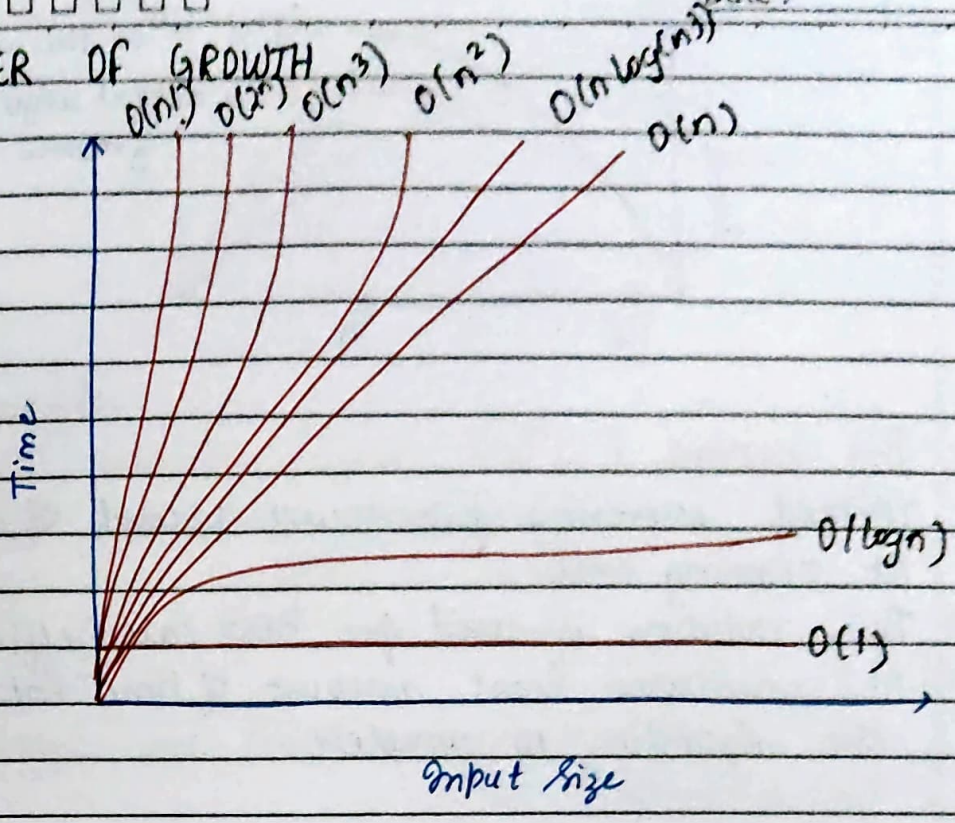
- O(1): hashing, array indexing
- O(log n): Binary Search
- O(n): linear Search
- O(n log n): Merge/Quick Sort
- O(n²): Bubble Sort / Selection sort
- O(n³): Matrix multiplication, floyd, Warshal
- O(2ⁿ): Tower of Hanoi / recursive fibonacci
- O(n!): Travelling Salesman prob

M T W T F S S

COMPASS

Date :

ORDER OF GROWTH



8m # ASYMPTOTIC NOTATIONS

There are three notations

(i) O (Big oh)

(ii) Ω (Big omega)

(iii) θ (Big theta)

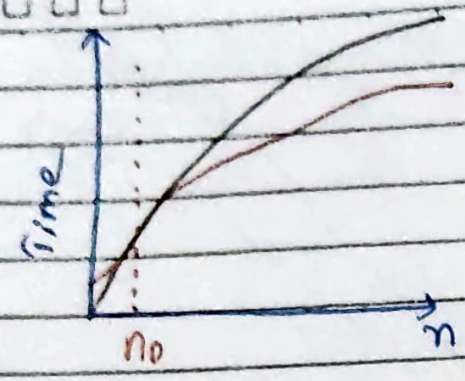
(i) Big oh (O)

> Method expressing the upper bound of an algorithm at running time.

> This notation is used for worst case efficiency.

> It measures longest amount of time taken by the algorithm to complete.

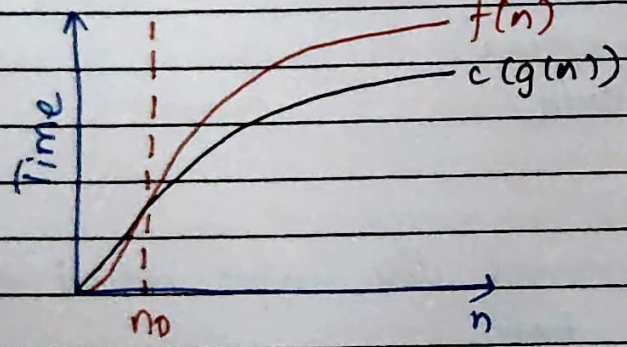
$$O(g(n)) = \{ f(n) : \text{there exist positive constant } c \text{ \& } n_0 \text{ such that } 0 \leq f(n) \leq c(g(n)) \text{ for all } n \geq n_0 \}$$



(ii) Big omega (Ω)

> Method expressing the lower bound of an algorithm at running time
> This notation is used for BEST case efficiency
> It measures least amount of time taken by the algorithm to complete.

$\Omega(g(n)) = \{ f(n)$: there exist positive constant c & no such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0 \}$



(iii) Big theta (θ)

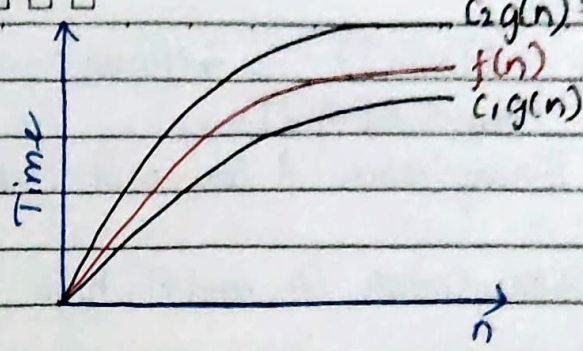
> Method expressing b/w upper bound and lower bound of an algorithm.
> This notation is used for avg case efficiency
> It measures the average amount of time taken by the algorithm to complete.

$\theta(g(n)) = \{ f(n)$: there exist positive constant c & no such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0 \}$

M T W T F S S

COMPASS

Date :



Examples:

wrt Linear Search

(i) Worst case time complexity

$T(n) \in O(n)$

(ii) Best case time complexity

$T(n) \in \Omega(1)$

(iii) Average case time complexity

$T(n) \in \Theta(n)$

BINARY SEARCH:

* Divide and conquer technique

Diagrammatic representation:

0	1	2	3	4	5	6	7	8	9
11	22	33	44	55	66	77	88	99	100

55

mid = (low + high)/2

5	6	7	8	9
66	77	88	99	100

mid = [(5+9)/2)

66	77	88	99	100

if (key == A[mid]) = 7

ALGORITHM BinarySearch (A[], low, high, key)

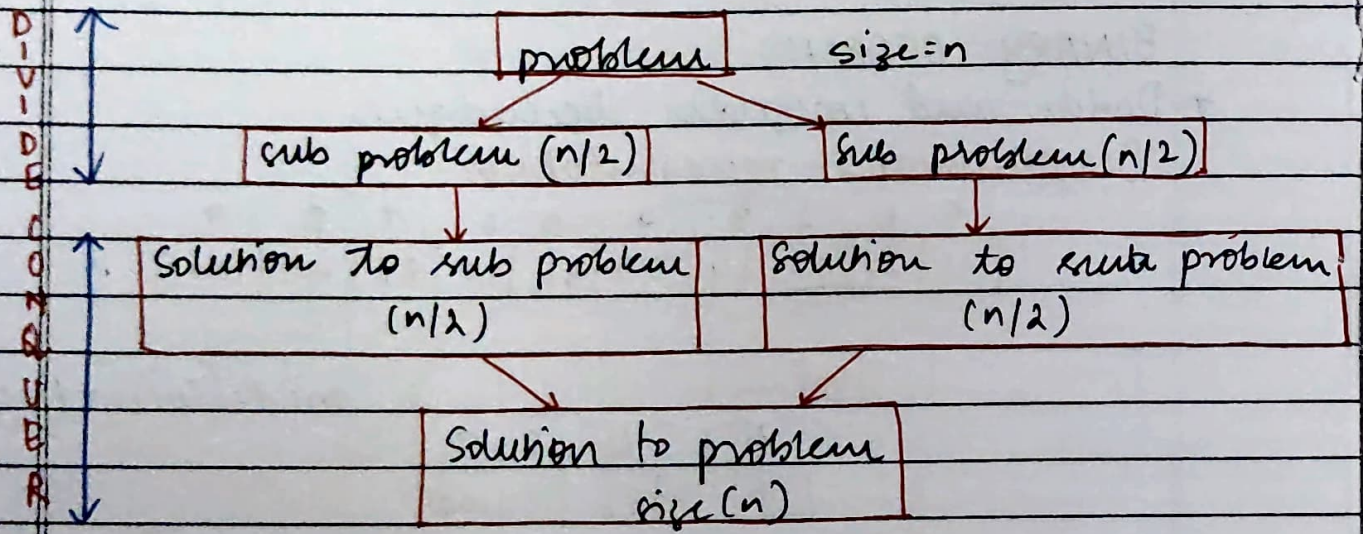
if (low > high) return -1

mid = [(low + high)/2]

```
if (key == A[mid])  return mid
else if (key < A[mid])
return BinarySearch (A, low, mid-1, key)
else
return BinarySearch (A, mid+1, high, key)
```

DIVIDE AND CONQUER

Divide and conquer is an algorithm design technique that divide a problem into smaller subproblem, solve them independently and combines their results to ~~solution~~ solve the original problem.



Examples:

1) Binary Search Algorithm
2) Merge sort Algorithm
3) Quick sort Algorithm
4) Strassens Matrix Multiplication.

M T W T F S S

→ stable sorting algorithm.

COMPASS

Date :

MERGE SORT:

* Divide and conquer Algorithm

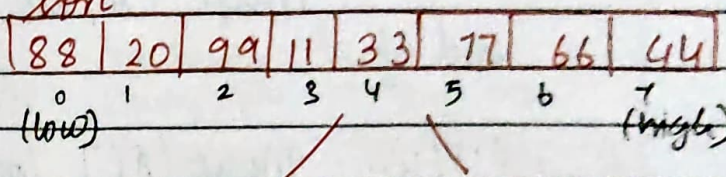
//time complexity :

Worst case : $T(n) \in O(n \log n)$

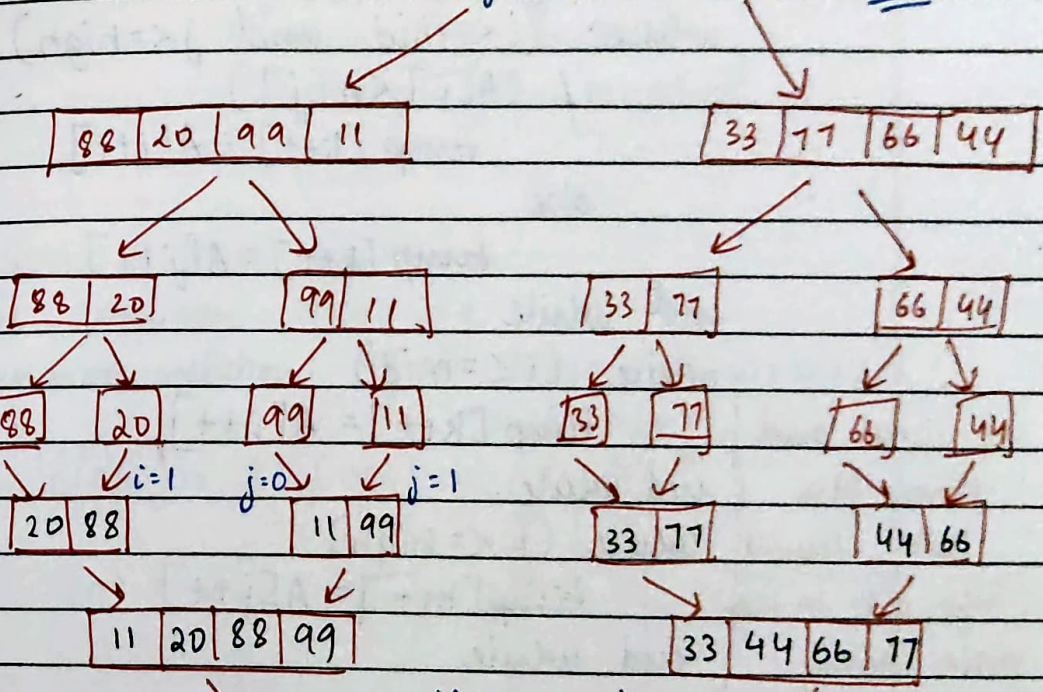
Best case : $T(n) \in \Omega(n \log n)$

Average case : $T(n) \in \Theta(n \log n)$

//apply merge sort



mid = ⌊(low+high)/2⌋ = ⌊(0+7)/2⌋ = 3



if true i++ (or) j++

if i false print j

if j false print i

while (i < j)

if (i < j) → print(i) → i++

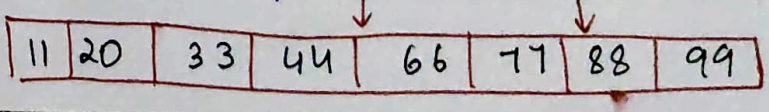
else → print(j) → j++

i++

while (i > j)

if (i > j) → print(j) → j++

else → print(i) → i++ j++



DIVIDE

CONQUER

sort + merge

M T W T F S S

Date :

```
//ALGORITHM MergeSort(A[], low, high)

    if (low < high)
        mid = ⌊(low + high)/2⌋
        MergeSort(A, low, mid)
        MergeSort(A, mid+1, high)
        Merge(A, low, mid, high)
    end if

ALGORITHM Merge(A[], low, mid, high)
    i = low, j = mid+1, k = low
    while (i<=mid and j<=high)
        if (A[i] < A[j])
            temp[k++] = A[i++]
        else
            temp[k++] = A[j++]
    end while
    { while (i<=mid)
        temp[k++] = A[i++]
    end while
    while (j <= high)
        temp[k++] = A[j++]
    end while }

    { for k=low to high do
        A[k] = temp[k]
```

to check and store the last element left out in the main array

print the final sorted list of elements from temp

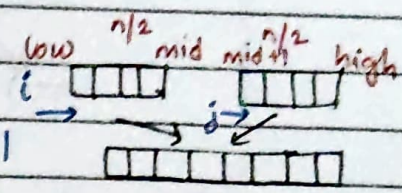
M T W T F S S

COMPASS

Date :

//time complexity analysis for merge sort.

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ T(n/2) + T(n/2) + cn & \end{cases}$$



- $T(n/2)$: recursively solving left subarray
- $T(n/2)$: recursively solving right subarray
- cn: time spent merging two subarrays

→ This can be solved by 3- methods

(i) Substitution method

(ii) Recursion - tree method

(iii)

(1) SUBSTITUTION METHOD.

Base condition: $T(1) = c$

Recursion relation: $T(n) = T(n/2) + T(n/2) + cn$

$$= 2T(n/2) + cn$$

$$T(n/2) = 2T(n/4) + c(n/2)$$

$$T(n) = 2\left[2T(n/4) + c(n/2)\right] + cn$$

$$= 2^2 T(n/4) + 2c(n/2) + cn$$

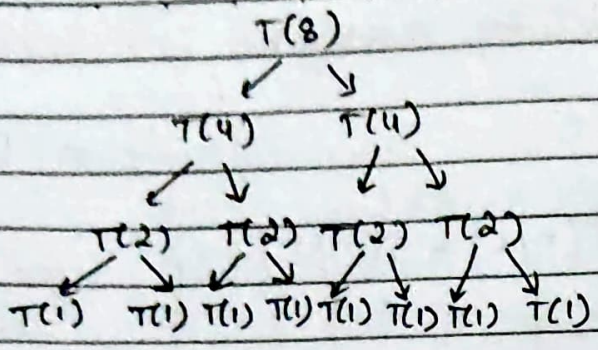
$$= 2^2 T(n/2^2) + 2c(n) \rightarrow (1).$$

$$T(n/2^2) = 2T(n/2^3) + c(n/2^2)$$

$$T(n) = 2^3 T(n/2^3) + 3c(n)$$

$$T(n) = 2^4 T(n/2^4) + 4c(n)$$

Similarly..... $\boxed{T(n) = 2^i T(n/2^i) + ic(n)}$



Base condition [stopping condition]

$T(1) = c$

$T(n/2i) = T(1)$

$n = 2^i \Rightarrow i = \log_2(n)$

~~$(n/2)i$~~ $n/2^i = 1$

$T(n) = 2^i T(n/2^i) + icn$

$T(n) = n\,T(n/n) + \log_2(n)\,c(n)$ $\quad T(1) = c$

$= n\,T(1) + \log_2 n\, cn$

$= nc + nc\log_2 n$

$T(n) = nc[1 + \log_2 n]$

if $n >>> 1$ ....

Time complexity :

$T(n) \in O(n \log_2 n)$ → WORST CASE

$T(n) \in \Omega(n \log_2 n)$ → BEST CASE

$T(n) \in \Theta(n \log_2 n)$ → AVERAGE CASE

Space complexity :

= input space + auxilary space

$= O(n) + O(n) + O(1) + O(1) + O(1)$

$= i + j + k$

constant!!

M T W T F S S

COMPASS

Date :

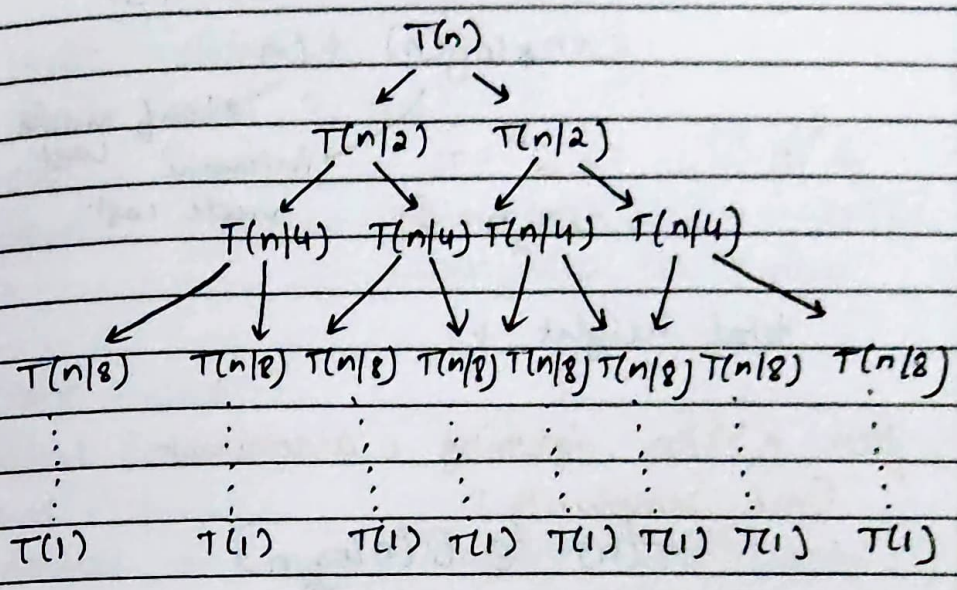
(2) RECURSION TREE METHOD

Recursion relation: $2T(n/2) + cn$

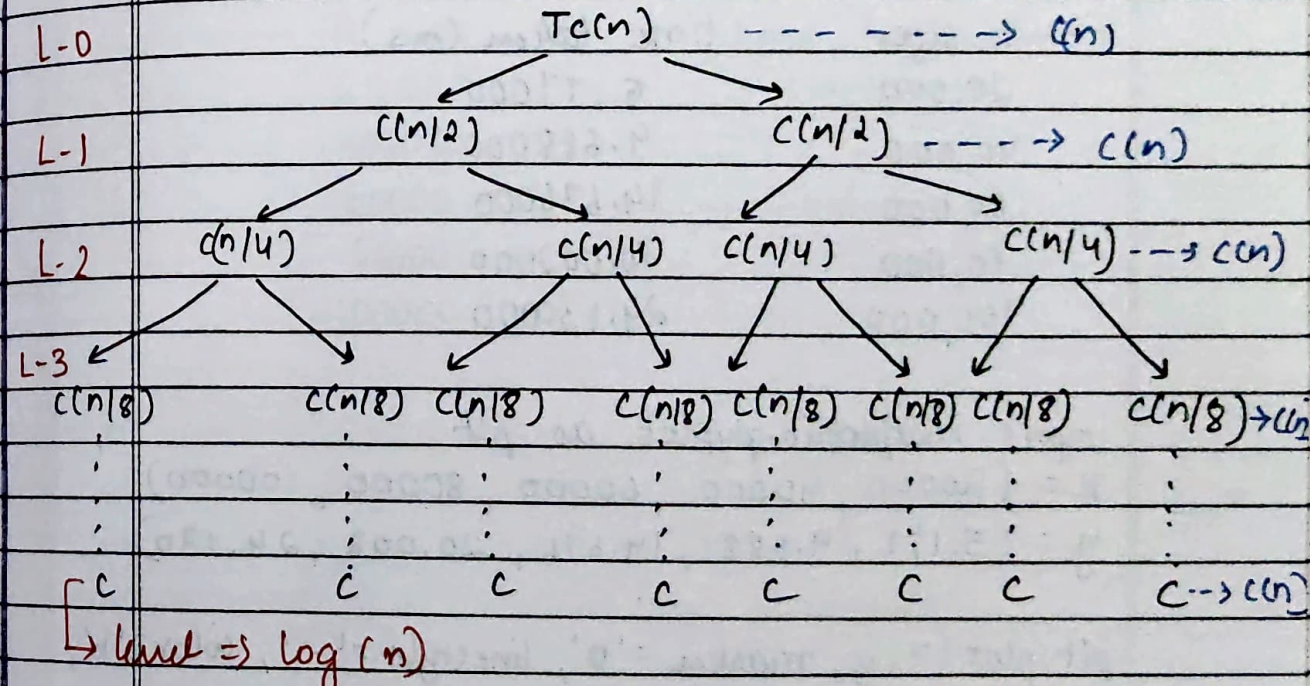
Recursion call for merge cost:

if not given take $T(1) = 1$

$T(1) = C$;



Recursion tree cost of merging



↳ Level $\Rightarrow \log(n)$

Total cost = sum of cost at each level

$$= \sum_{0}^{\text{levels}} \text{cost}$$

$$= cn + cn + cn + \dots + cn$$

$$= cn * \log(n) + cn$$

→ leaf node cost

→ internal node cost.

$2^h/n = 1 \Rightarrow 2^h = n$

$\log_2 n = h$

total height + 1

for $n >> c$ ignoring c as constant

Time complexity:

$T(n) \in O(n \log_2 n)$

GRAPH OF MERGE SORT:

n. size	time taken (ms)
20,000	5.177000
40,000	9.688000
60,000	14.676000
80,000	20.002000
100,000	24.130000

```
import matplotlib.pyplot as plt
x = (20000, 40000, 60000, 80000, 100000)
y = (5.177, 9.688, 14.676, 20.002, 24.130)

plt.plot(x, y, marker='o', linestyle='-', color='b',
    markersize=8, linewidth=2)
plt.xlabel('Input Size (n)')
plt.ylabel('Time (ms)')
```

merge sort : 227362
compare = 971341

M T W T F S S

COMPASS

Date :

```
plt.title('Merge sort performance')
plt.grid(True)
plt.show()
```

```
start =clock();
divide (arr1, 0, n-1);
end = clock();
double m_time = ((double)(end - start))/
                CLOCKS_PER_SEC *1000;
```

```
start = clock();
Bsort (arr, 2, n);
end = clock();
double b_time = ((double)(end - start)) /
                CLOCKS_PER_SEC *1000;
```

n size	time taken (ms)
20,000	
40000	
60000	
80000	
100000	

(3) MASTER THEOREM:

The method used to determine the asymptotic time complexity of divide and conquer recurrence relation of the form:

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$: Number of subproblems
$b > 1$: factor by which input size is divided
$f(n)$: cost of dividing and merging result

Compare $f(n)$ and $n^{\log_b a}$

Case 1: If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \theta(n^{\log_b a})$ $(fn < n^{\log_a b})$

Case 2: $f(n) = \theta(n^{\log_b a})$ then $T(n) = \theta(n^{\log_b a} \log_2 n)$

Case 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$ and if $a + (n/b) \leq c f(n)$ for some constant $c < 1$ & all sufficient n, then

$$T(n) \in \theta(f(n)).$$

1) Merge sort time complexity evaluating by using Master theorem.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

M T W T F S S

COMPASS

Date :

Recursive relation:

$T(n) = 2T(n/2) + n$

Master theorem recursive relation:

$T(n) = aT(n/b) + f(n)$

Step:1 Identify a, b and f(n)

$a = 2$

$b = 2 \qquad f(n) = n$

Step:2 Compare f(n) and $n^{\log_b a}$

$f(n) = n$

$n^{\log_b a} = n^{\log_2 2} = n^1$ $\qquad [\log_2 2 = 1]$

$\therefore n^{\log_b a} = n$

if $(f(n) == n^{\log_b a})$ then master theorem — CASE:2

Case:2

$T(n) = \theta(n^{\log_b a} \log n)$

$T(n) = \theta(n^{\log_2 2} \log n)$

$T(n) = \theta(n^1 \log n)$

$\therefore \boxed{T(n) = \theta(n \log n)}$

Time complexity....

$T(n) = \theta(n \log n)$

$T(n) = O(n \log n)$

$T(n) = \Omega(n \log n)$

M T W T F S S

Date :

2) Solve recursive relation using Masters theorem

$T(n) = 9\,T(n/3) + n$ (a = 9, b = 3, f(n) = n)

Recursion relation: $T(n) = 2T(n/2) + f(n)$

Step:1 Identify a, b, f(n)

$a = 9$

$b = 3$

$f(n) = n$

Step:2 Compare $f(n)$ and $n^{\log_b a}$

$n^{(\log_b a)} = n^{\log_3 9} = n^2$

$\log_3 3^2 = 2\log_3 3 = 2$

if $(f(n) < n^{\log_b a})$

$(n < n^2)$ $\therefore \rightarrow$ CASE : 1

time complexity:

$T(n) = \theta(n^{\log_b a})$

$= \theta(n^2)$

$T(n) \in \theta(n^2)$

3) Solve recursive relation using Master's theorem

$T(n) = 3T(n/4) + n\log n$ (a = 3, b = 4, f(n) = n log n)

Step:1 Identify a, b, f(n)

$a = 3$ $f(n) = n\log n$

$b = 4$

M T W T F S S

COMPASS

Date :

use calculator

$n^{\log_b a} = n^{\log_4 3}$

$= n^{0.7925}$

Step:2 (compare with f(n))

$f(n) = n \log n$

$n^{\log_b a} = n^{0.7925}$

33.219
6.

589

Substitute some value.

$n \log_2 n$

$\boxed{n \log n > n^{0.7925}}$

$f(n) > n^{\log_b a}$ → CASE:3

time complexity:

$f(n) = n \log n$

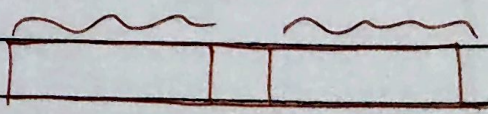
$T(n) = \Theta(f(n))$

$T(n) \in n \log n$ → Ans.

4) Solve binary search worst time complexity using Master's Theorem

$$T(n) = \begin{cases} 1 & \text{if } (n \leq 1) \\ T(n/2) + 1 & \text{if } (n > 1) \end{cases}$$

time spent for comparision a[i] == key



if n = 1

Time Solver

- right subarray
- left subarray

COMPASS

M T W T F S S

Date :

Step 1: Identify a, b, f(n)

$a = 1$

$b = 2$

$f(n) = 1$

Step 2: Compare f(n) & $n^{\log_b a}$

$f(n) = 1$

$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$

$f(n) = n^{\log_b a}$ → CASE : 2

$T(n) = O(n^{\log_b a} \log_2 n)$

$= O(n^{\log_2 1} \cdot \log_2 n)$

$= O(1 \cdot \log_2 n)$

∴ time complexity Binary Search worst case

$T(n) \in O(\log_2 n)$

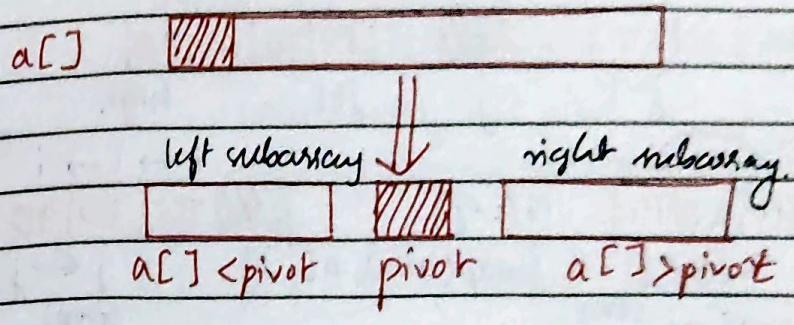
M T W T F S S

COMPASS

Date:

QUICK SORT:

Quick sort in divide and conquer algorithm. That select a pivot element, partition the array and sort subarray recursively.



Time complexity

Best case time complexity = $\Omega(n \log n)$

Average case time complexity = $\theta(n \log n)$

Worst case time complexity = $O(n^2)$

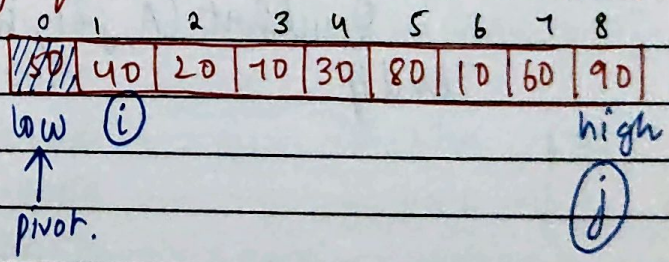
Quick sort selection:

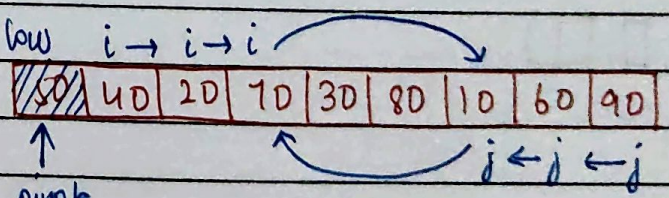
(i) Choose pivot first element array

(ii) Choose pivot last element array

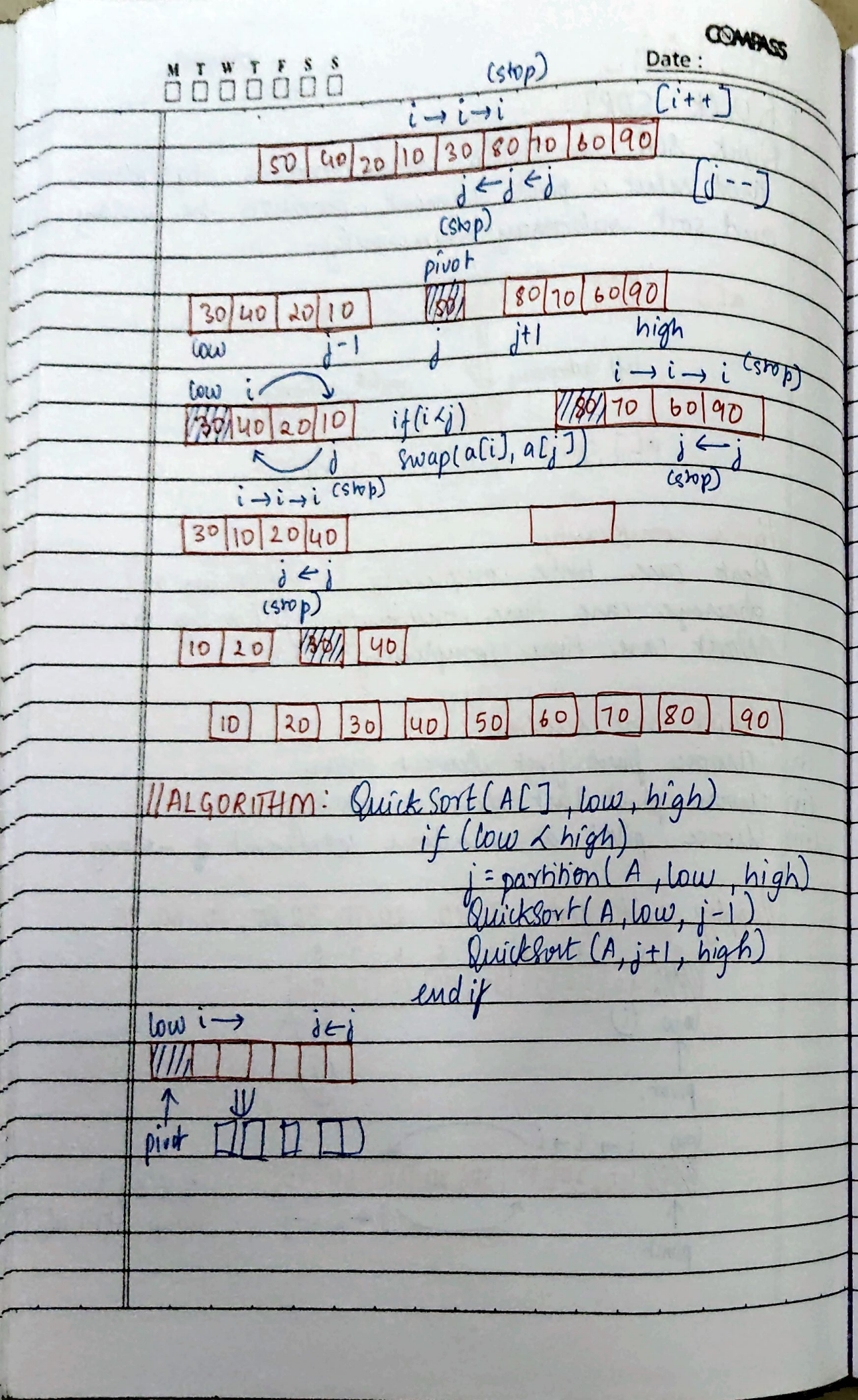
(iii) Choose pivot as random element of array.

Apply quick sort 50, 40, 20, 70, 30, 80, 10, 60, 90





if (i<j)
swap(a[i], a[j])


M T W T F S S
Date :
(stop)
i → i → i
[i++]
50 40 20 10 30 80 70 60 90
j ← j ← j
[j--]
(stop)
pivot
30 40 20 10
50
80 70 60 90
low
j-1
j
j+1
high
low i
i → i → i (stop)
30 40 20 10
if(i < j)
swap(a[i], a[j])
80 70 60 90
j
j ← j
(stop)
i → i → i (stop)
30 10 20 40
j ← j
(stop)
10 20 30 40
10 20 30 40 50 60 70 80 90
//ALGORITHM: QuickSort(A[], low, high)
if (low < high)
j = partition(A, low, high)
Quicksort(A, low, j-1)
Quicksort(A, j+1, high)
endif
low i →
j ← j
pivot

```
partition(A[], low, high)
    pivot = a[low]
    i = low+1
    j = high
    while (i ≤ j)
    {   while (a[i] ≤ pivot)   i++
        while (a[j] ≥ pivot)   j--
        if(i<j)  swap(a[i], a[j])
    endwhile
    if(i>j)  swap(a[j], pivot)
     return j
```

//graph

n (size)	time (ms)
100000	3.49
20000	7.21
30000	4.8
40000	6.69
50000	6.078

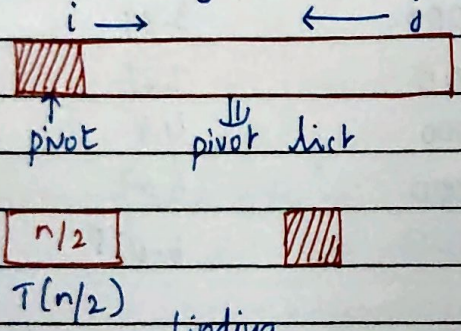
n(size)	normal q-sort	descending q-sort
10000	4.149000 ms	37.439
20000	8.288	135.858
30000	12.774	257.569
40000	6.634	446.766
50000		

10000	→	1.466	→	28.199	→	1.582
20000	→	3.234	→	108.305	→	3.110
30000	→	4.885	→	212.296	→	4.841
40000	→	7.374	→	384.960	→	6.481
50000	→	9.476	→	589.556	→	8.732

Time complexity → Quick Sort

Basic operation numbers of comparition pivot position.

Best Case:
Pivot divide array into equal halves pivot divide the array into 2 equal sub problems



time spent ~~for doing~~ finding pivot position (no. of comparisions.

$$T(n) = \begin{cases} c & \text{if } n=1 \\ 2 + \left(\frac{n}{2}\right) + cn & \end{cases}$$

Time spent algorithm dividing 2 sub problems.

Number of comparisions required for partition

M T W T F S S

COMPASS

Date :

(1) RECURSION RELATION

$$T(n) = 2T(n/2) + cn$$

$$T(n) = 2\left[2T(n/2) + cn/2\right] + cn$$

$$T(n) = 2^2 T(n/2^2) + 2cn$$

$$T(n) = 2^3 T(n/2^3) + 3cn$$

$$\vdots$$

$$\boxed{T(n) = 2^i T(n/2^i) + icn}$$

Base condition $T(1) = c$

$$T(n/2^i) = T(1)$$

$$n/2^i = 1 \Rightarrow n = 2^i$$

$$i = \log_2 n$$

$$T(n) = 2^i T(n/2^i) + icn$$

$$= nT(1) + \log_2 n \, cn$$

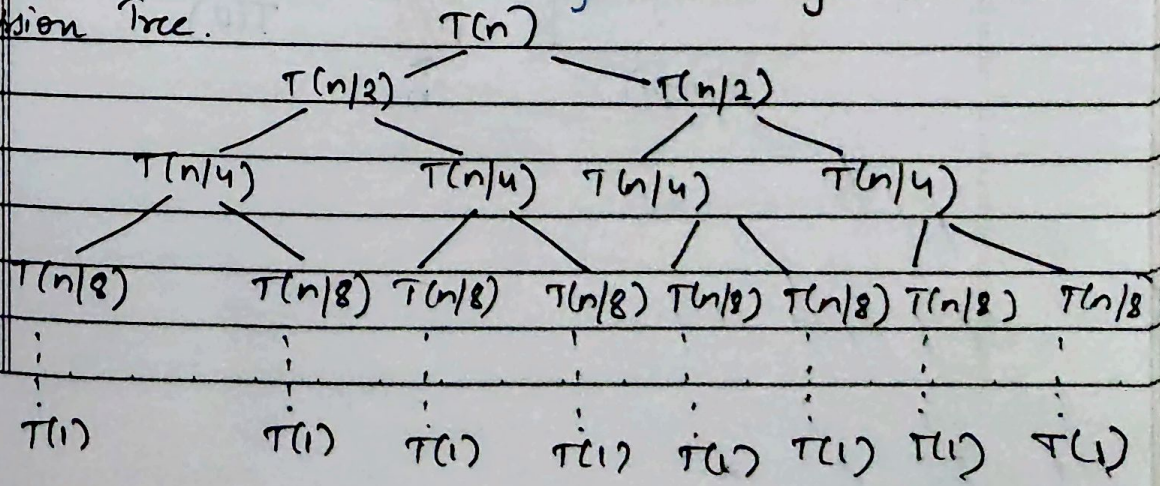
$$T(n) = cn + cn\log_2 n$$

Let $n >>> 1$, ignore the constant c

$\boxed{T(n) \in \Omega(n\log_2 n)}$ → Best case

$T(n) \in \Theta(n\log_2 n)$ → Avg case

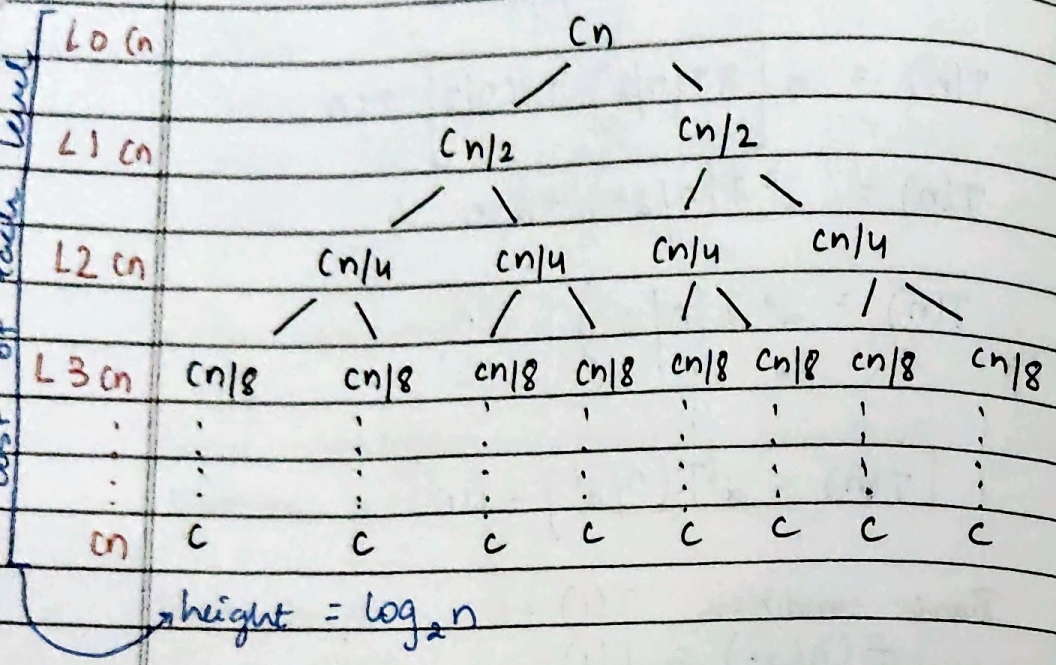
Recursion Tree.



M T W T F S S

Date :

COST OF PARTITION:



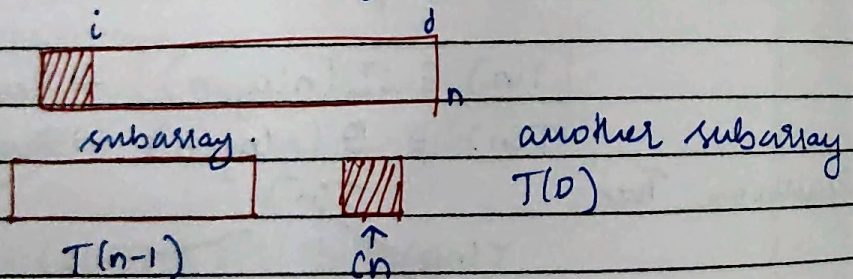
$T(n) = cn \log_2 n + cn$ level

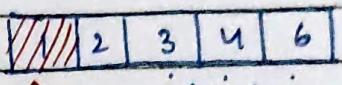
Total cost partition = $\sum_{0}^{} cn$

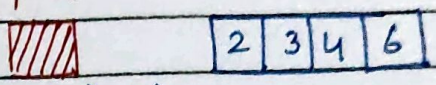
Worst Case:

→ Worst case happens when the input is already sorted in ascending (or) descending list

→ pivot divides the array unbalanced partition







$$T(n) = \begin{cases} c & \text{if } (n=1) \\ T(n-1) + T(0) + cn & \text{if } n>1 \end{cases}$$

core subarray size $(n-1)$

another subarray (empty)

number of comparison portion (cn)

Solve: $T(n) = T(n-1) + cn$

Base condition $T(1) = c$

COMPASS

M T W T F S S

Date :

RANDOMIZED QUICK SORT

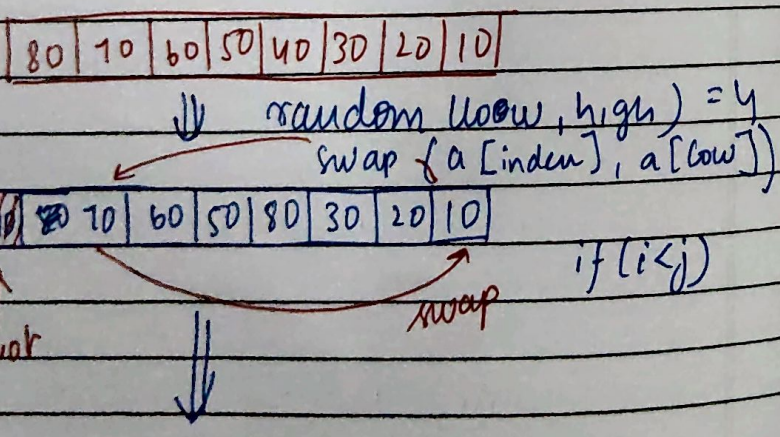
Randomized Quick sort in which the pivot element is chosen randomly to reduce the probability of worst case partitioning and achieve expected time complexity $O(n \log n)$

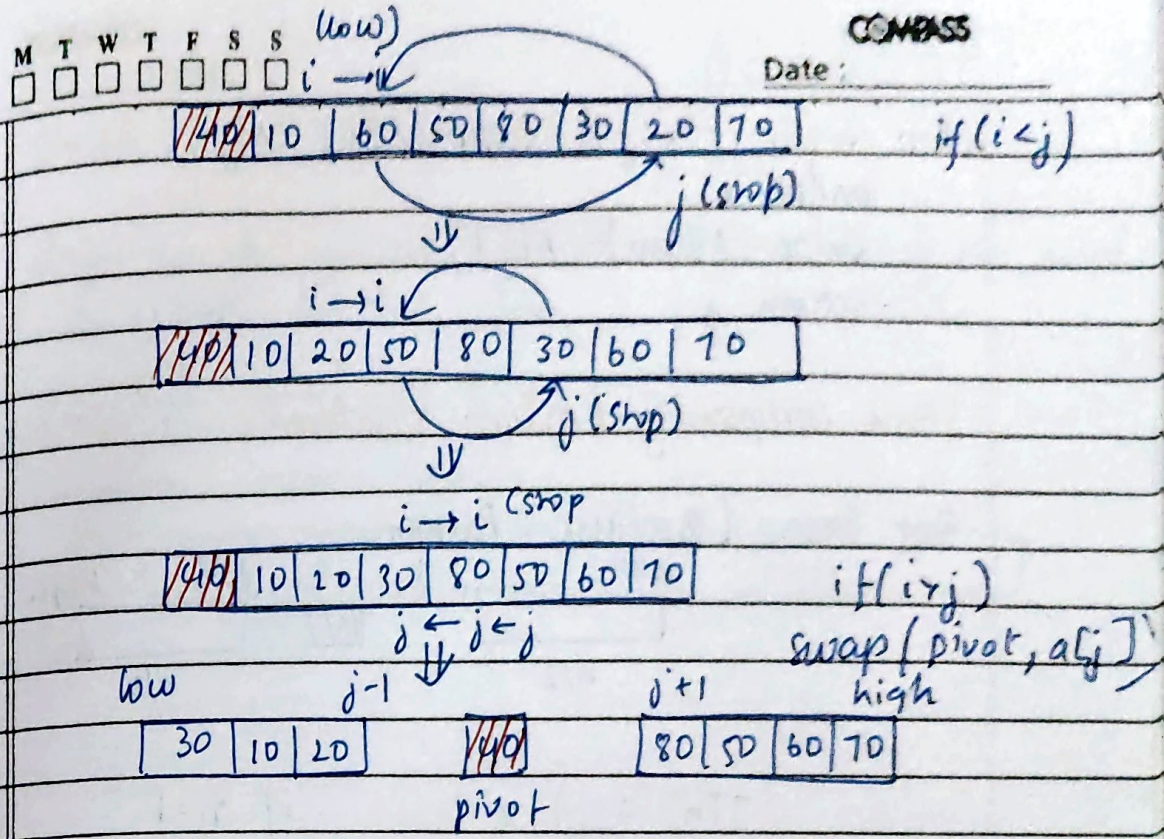
Normal Quick Sort	Randomized Quick Sort.
• Fixed pivot	• Random pivot
• Performance of algorithm degrades for sorted / almost sorted inputs	• Performance of algo does not depend on input (sorted, not sorted)
• Worst case more likely $T(n) \in O(n^2)$	• Worst case very rare $T(n) \in O(n^2)$

→ Most of the time Random Quick sort gives time complexity $\theta(n \log n)$.

Apply Random Quick Sort

80, 70, 60, 50, 40, 30, 20, 10





ALGORITHM RandomQuickSort (A, low, high)

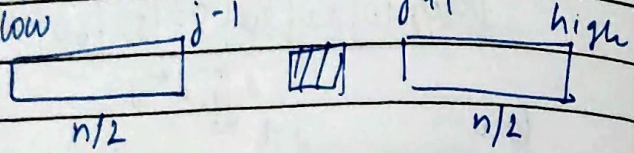
```
    if (low < high)
        j = RandomPartition (A, low, high)
        RandomQuickSort (A, low, j-1)
        RandomQuickSort (A, j+1, high)
    endif
RandomPartition (A, low, high)
    i = random (low, high)
    swap (A[i], A[low])
    j = partition (A, low, high)
    return j

Partition (A, low, high)
    pivot = A[low]
    i = low + 1
    j = high
    while (i ≤ j)
        while (A[i] ≤ pivot)  i++;
        while (A[j] > pivot)  j--;
```

```
        if(i<j) swap (A[i], A[j])
endwhile
swap (A[low], A[j])
return j
```

// time complexity Random Quicksort

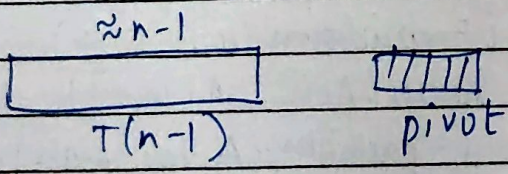
Best case [Balance Position]



$$T(n) = \begin{cases} c & \text{if } n=1 \\ T(n/2) + T(n/2) + cn & \text{if } n>1 \end{cases}$$

$T(n) \in \Omega(n \log n)$ → Best case time
$T(n) \in \Theta(n \log n)$ → Average case time

> Worst case time complexity very rare - imbalanced position.



$$T(n) = \begin{cases} c & \text{if } n=1 \\ T(n-1)+cn & \text{if } n>1 \end{cases}$$

$T(n) = O(n^2)$ Worst case
(less probability)